Technology comparison · October 12, 2026

Frontend Framework Comparison

Angular vs React vs Svelte vs Vue vs Blazor: architecture, rendering, performance, developer experience, ecosystem, enterprise concerns, and npm download trends.

Angular React Svelte Vue Blazor

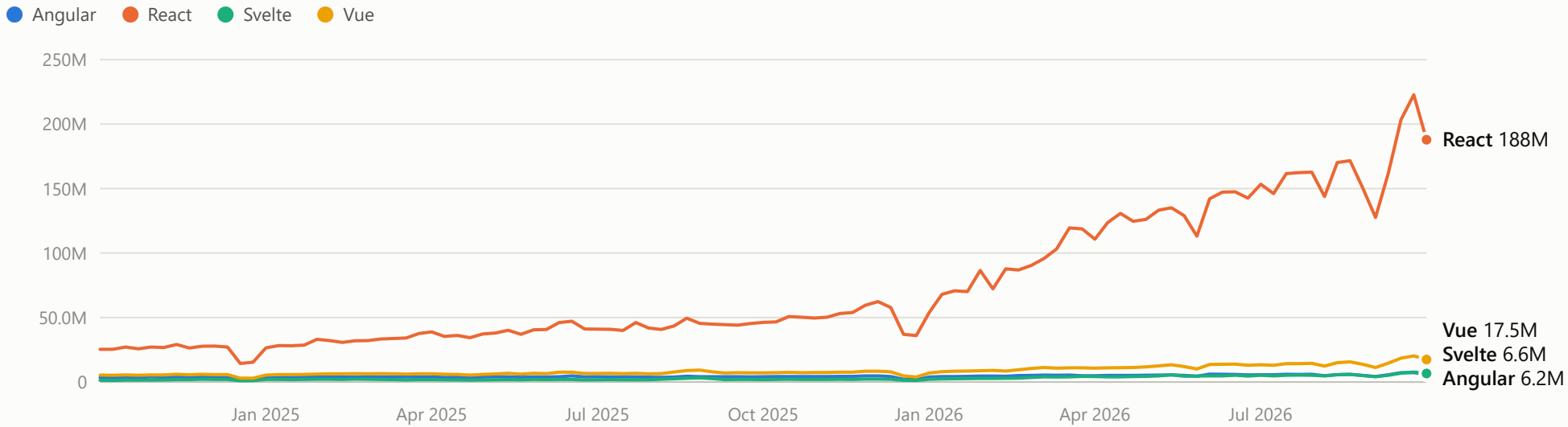
React weekly npm downloads	Vue weekly npm downloads	Svelte / Angular weekly	Blazor WebAssembly (NuGet, all-time)
188M	17.5M	6.6M / 6.2M	77.3M
about 11× Vue	+148% vs a year ago	close to each other	NuGet has no weekly series

Download and repository figures were pulled live on Oct 12, 2026 from api.npmjs.org, registry.npmjs.org, api.github.com and the NuGet search API. Bundle sizes and 1–5 scores are approximate and meant for comparison only.

npm download trends

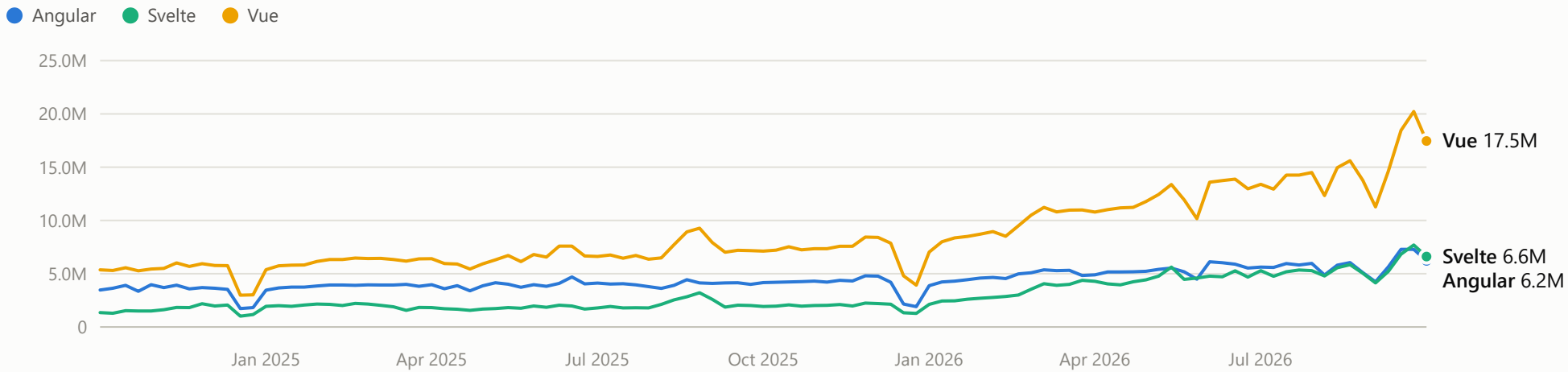
Weekly npm downloads, Oct 2024 – Oct 2026

Core package per framework: `@angular/core`, `react`, `svelte`, `vue`. Blazor ships on NuGet, so it isn't in npm data.



Same data without React

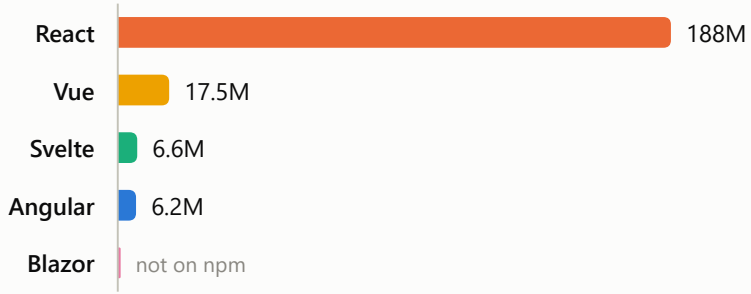
React's volume flattens the other three on a shared scale, so this view uses its own axis.



npm and GitHub at a glance

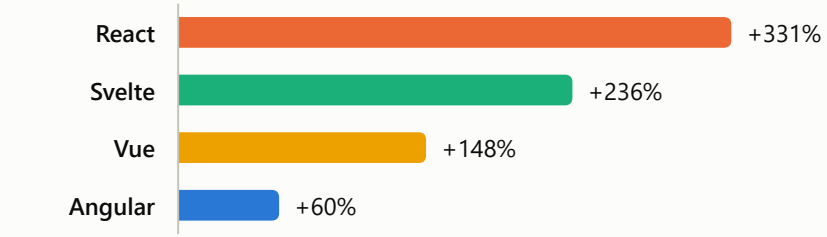
Downloads in the last week

Oct 4–10, 2026



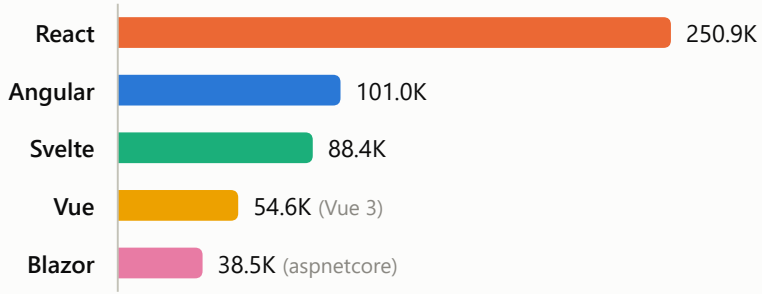
Download growth

Last 4 weeks vs the same 4 weeks one year earlier



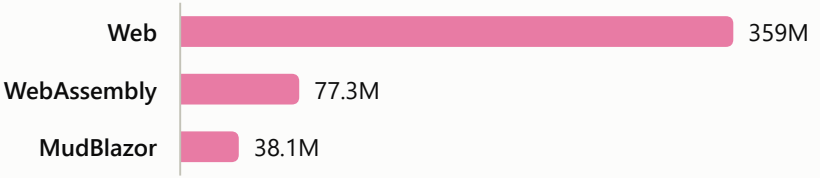
GitHub stars

Main repository of each framework



Blazor on NuGet: total downloads

All-time totals of `Microsoft.AspNetCore.Components.Web`, `.WebAssembly` and MudBlazor (top UI library)



React's npm count is inflated: it's also installed transitively by many tools and React Native projects, so downloads measure installs, not apps. GitHub stars for Vue cover only the Vue 3 repo (the Vue 2 repo, vuejs/vue, has its own stars), and Blazor's repo is all of ASP.NET Core.

Package and repository stats

Metric	Angular	React	Svelte	Vue	Blazor
Package	`@angular/core`	`react`	`svelte`	`vue`	`Microsoft.AspNetCore.Components.*` (NuGet)
Weekly downloads (Oct 4–10, 2026)	6,207,498	187,877,532	6,610,438	17,460,488	n/a: NuGet publishes totals only
Total downloads	—	—	—	—	WebAssembly: 77,324,125 Web: 359,339,220
Growth (last 4 wks vs a year ago)	+60%	+331%	+236%	+148%	—
Latest version	22.2.2	19.3.0	5.57.2	3.5.43	10.0.12
Last published	2026-10-08	2026-09-09	2026-10-06	2026-09-17	—
Package age	10.5 yrs	15.0 yrs	9.9 yrs	12.8 yrs	8.7 yrs (first preview)
Unpacked size (core package)	7.06 MB	179 KB	2.95 MB	2.55 MB	—
Direct dependencies	1	0	15	5	—
GitHub repo	`angular/angular`	`facebook/react`	`sveltejs/svelte`	`vuejs/core`	`dotnet/aspnetcore`
GitHub stars	101,032	250,897	88,443	54,630	38,469
GitHub forks	30,115	51,479	7,929	9,248	13,529
Open issues + PRs	1,185	1,432	1,141	905	4,230
Last commit	2026-10-12	2026-10-09	2026-10-12	2026-10-12	2026-10-12

*Versions as published on Oct 12, 2026. Open issues on GitHub include pull requests.

Comparison charts: scores

Score heatmap (1–5)

From the scorecard in section 8. Mixed Blazor cells use the C#-team reading: bundle size 3 (average of WASM 2 and SSR 4), learning curve 5.

Criterion	Angular	React	Svelte	Vue	Blazor
Performance	4	4	5	5	3
Bundle size / first load	3	3	5	4	3
Learning curve	2	3	5	4	5
Ecosystem	4	5	2	4	3
Enterprise structure	5	3	3	3	5
Long-term stability / LTS	5	4	3	3	5
Hiring pool (global)	4	5	2	3	2
Full-stack code sharing	1	2	2	2	5

1 5 (higher is better)

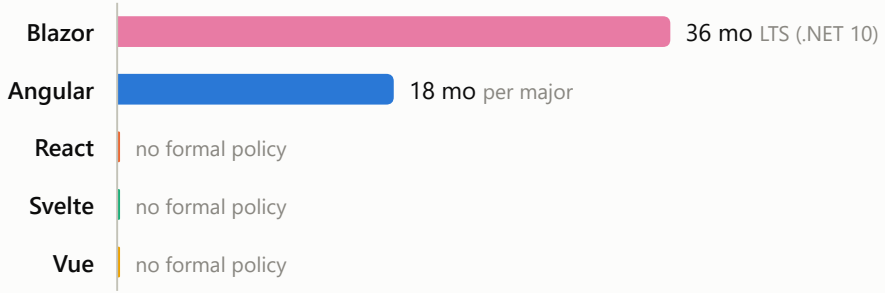
Total score

Sum of the 8 criteria above, out of 40



Support window per release

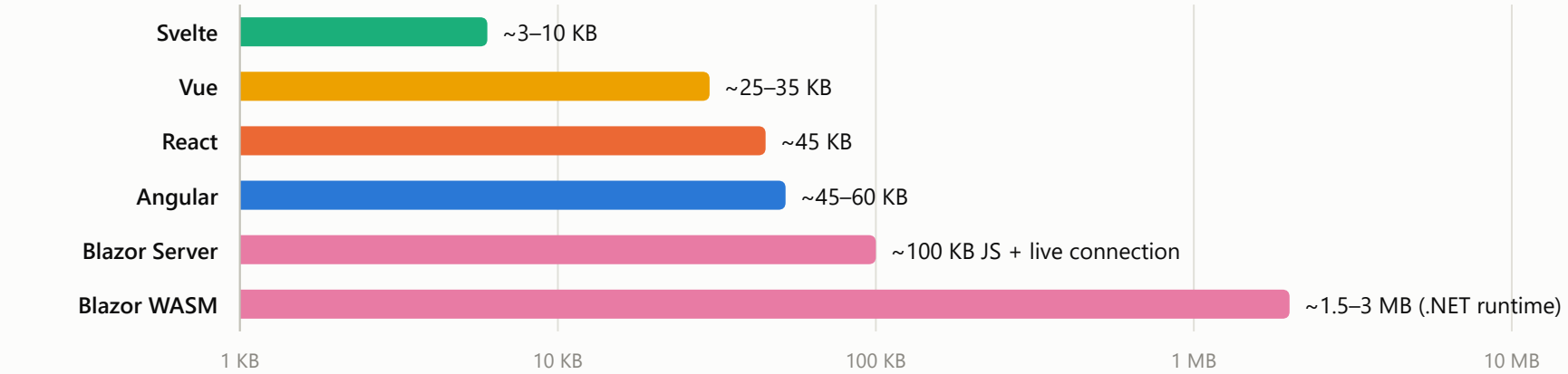
Months of official support for each major version



Comparison charts: size and built-in features

Initial download size (gzip, log scale)

Each gridline is 10× the previous one. Blazor WebAssembly is roughly 40× Angular and 300× Svelte.



What comes in the box

Features available without picking a third-party library, from sections 2, 3 and 7

Feature	Angular	React	Svelte	Vue	Blazor
Routing	●	○	◑	◑	●
Forms + validation	●	○	○	○	●
HTTP client	●	○	◑	○	●
Dependency injection	●	○	○	◑	●
State management	●	◑	●	◑	◑
Scoped styles	●	○	●	●	●
Animations	●	○	●	●	○
i18n	●	○	○	◑	●
SSR	●	◑	◑	◑	●
PWA / offline	●	○	○	○	◑
Built-in score (of 10)	**10**	**1**	**4.5**	**4.5**	**8**

● built in ◑ official library / meta-framework ○ third-party only

1. Identity & fundamentals

Aspect	Angular	React	Svelte	Vue	Blazor
Type	Full framework, everything built in	UI library	Compiler plus framework	Progressive framework	.NET web UI framework
Backed by	Google	Meta	Vercel and community	Community (Evan You), sponsors	Microsoft
First release	2016 (AngularJS 2010)	2013	2016	2014	2018 (GA 2020)
Latest version*	22.2.2	19.3.0	5.57.2	3.5.43	.NET 10 (10.0.12)
Language	TypeScript (required)	JS / TS (JSX/TSX)	JS / TS	JS / TS	**C#** with Razor
License	MIT	MIT	MIT	MIT	MIT
Current style	Standalone components, Signals, zoneless	Function components, Hooks, Server Components, React Compiler	Runes (`$state` , `$derived`)	Composition API, `<script setup>`	Razor components, "Blazor Web App"
How opinionated	Very high	Low	Medium	Medium	High

2. Architecture & core concepts

Aspect	Angular	React	Svelte	Vue	Blazor
Rendering engine	Ivy (incremental DOM)	Virtual DOM	No VDOM: compiled into direct DOM updates	Virtual DOM tuned by the compiler (Vapor mode coming)	Render-tree diffing in .NET
Reactivity	Signals (RxJS also available)	Re-render plus hooks; compiler auto-memoizes	Fine-grained, compile-time runes	Fine-grained proxies (`ref`, `reactive`)	Event-driven re-render via `StateHasChanged`
Component format	Class + decorator + HTML template	Function returning JSX	`.svelte` single file	`.vue` single file	`.razor` file (markup + `@code`)
Data binding	Two-way (`[(ngModel)]`, `model()`)	One-way (controlled inputs)	Two-way (`bind:value`)	Two-way (`v-model`)	Two-way (`@bind`)
Dependency injection	✅ Built-in, hierarchical	❌ (Context only)	❌ (Context only)	Partial (`provide` / `inject`)	✅ Built-in, standard .NET DI
Routing	✅ Built-in	❌ Separate library (React Router, TanStack)	Through SvelteKit	Vue Router (official)	✅ Built-in
Forms & validation	✅ Reactive and template forms	❌ Library (React Hook Form, Formik)	Manual, or Superforms	Manual, or VeeValidate / FormKit	✅ `EditForm` + DataAnnotations
HTTP client	✅ `HttpClient` + interceptors	❌ fetch / axios / TanStack Query	fetch / SvelteKit `load`	fetch / axios	✅ `HttpClient`; Server mode calls services directly
State management	Signals, services, NgRx	Redux, Zustand, Jotai, TanStack	Runes and stores, built in	Pinia (official)	Scoped/singleton services, Fluxor
Style scoping	✅ Emulated encapsulation	❌ CSS Modules / CSS-in-JS / Tailwind	✅ Scoped by default	✅ `<style scoped>`	✅ CSS isolation (`.razor.css`)
Animations	✅ Built-in module	Library (Framer Motion)	✅ Built-in transitions	✅ `<Transition>`	CSS or JS interop

3. Rendering modes & meta-frameworks

Aspect	Angular	React	Svelte	Vue	Blazor
Meta-framework	Built in (`@angular/ssr`), Analog	Next.js, React Router 7 (Remix)	SvelteKit	Nuxt	Built in (Blazor Web App)
Client-side rendering	✅	✅	✅	✅	✅ WebAssembly
Server-side rendering	✅	✅	✅	✅	✅ Static SSR
Static site generation	✅ Prerender	✅	✅	✅	⚠️ Limited
Hydration	✅ Incremental hydration, `@defer`	✅ Streaming, Suspense	✅	✅ Lazy hydration	Not hydration: switches to interactive mode
Server Components	❌	✅ RSC	❌	❌	**Interactive Server**: UI runs on server over SignalR
Unique mode	—	—	—	—	**Auto**: starts on Server, moves to WASM once downloaded

4. Performance

Aspect	Angular	React	Svelte	Vue	Blazor
Baseline bundle (gzip)	~45–60 KB	~45 KB (react + react-dom)	**~3–10 KB**	~25–35 KB	WASM: **~1.5–3 MB** (.NET runtime); Server: ~100 KB JS
Runtime speed	Very good (best zoneless + signals)	Good (better with compiler)	**Excellent**	Excellent	WASM: fair to good; Server: network latency bound
First load	Good	Good	**Best**	Very good	WASM: **slowest**; Server and SSR: fast
Memory use	Medium	Medium	Low	Low	WASM: high; Server: memory per user on the server
Scaling cost	Static hosting / CDN	Static / edge	Static / edge	Static / edge	Server mode: one live connection per user + sticky sessions
AOT / tree-shaking	✅ AOT by default	✅	✅ Compiler	✅	✅ IL trimming + WASM AOT (bigger download)

5. Developer experience

Aspect	Angular	React	Svelte	Vue	Blazor
Learning curve	🔴 Steep	🟡 Medium (hooks are subtle)	🟢 **Easiest**	🟢 Easy	🟢 Easy for C# devs, 🔴 for JS devs
CLI / scaffolding	✅ Angular CLI (excellent)	Vite, Next CLI	`sv` CLI	`create-vue`, Nuxi	`dotnet new`, Visual Studio
Build tool	esbuild + Vite (dev)	Vite / Turbopack	Vite	Vite	MSBuild
Type safety	Native TypeScript	Excellent	Excellent	Excellent	**C# type system**, end to end
Hot reload	✅ Fast	✅ Fast Refresh	✅	✅	⚠️ Works, sometimes needs a restart
IDE	VS Code, WebStorm	VS Code, WebStorm	VS Code	VS Code (Vue - Official)	**Visual Studio, Rider**, VS Code (C# Dev Kit)
Debugging	Browser DevTools + Angular DevTools	React DevTools	Svelte DevTools	Vue DevTools (excellent)	Visual Studio debugger, including WASM
Unit testing	Vitest / Jest, TestBed	Vitest / Jest + RTL	Vitest	Vitest + Vue Test Utils	**bUnit**, xUnit / NUnit
E2E testing	Playwright / Cypress	Playwright / Cypress	Playwright	Playwright / Cypress	Playwright for .NET
Code sharing with backend	❌ (unless Node backend)	❌ (unless Node)	❌	❌	✅ **Shares DTOs, validation and logic with ASP.NET**

6. Ecosystem

Aspect	Angular	React	Svelte	Vue	Blazor
Ecosystem size	Large	**Largest**	Small, growing	Large	Medium, mostly commercial vendors
Libraries	Large	**Huge**	Small; any vanilla JS library works	Large	Small, plus all of .NET and JS interop
UI component libraries	Angular Material, PrimeNG, Taiga	MUI, Ant Design, shadcn/ui, Mantine	shadcn-svelte, Skeleton, Flowbite	Vuetify, PrimeVue, Quasar, Element Plus	**MudBlazor**, Radzen, Fluent UI Blazor
Enterprise grids / suites	DevExpress, Syncfusion, Kendo, AG Grid	All vendors, AG Grid	AG Grid (vanilla), Syncfusion	DevExpress, Syncfusion, Kendo, AG Grid	**DevExpress, Telerik, Syncfusion**, Radzen
Mobile	Ionic, NativeScript, Capacitor	**React Native** (best)	Capacitor	Ionic, Quasar, Capacitor	**.NET MAUI Blazor Hybrid**
Desktop	Electron, Tauri	Electron, Tauri	Electron, Tauri	Electron, Tauri	MAUI, WPF / WinForms BlazorWebView, Photino
Job market (global)	Large (enterprise)	**Largest**	Small	Medium (strong in Asia)	Niche (.NET shops)
Developer satisfaction	Medium, rising	High	**Highest**	High	High among .NET devs

7. Enterprise concerns

Aspect	Angular	React	Svelte	Vue	Blazor
Release cycle	Major every 6 months	Irregular	Irregular	Irregular	Yearly with .NET (every November)
LTS / support policy	✅ 18 months per major	❌ No formal LTS	❌ None	❌ None (Vue 2 ended 2023)	✅ **LTS 3 years**, STS 2 years
Upgrade tooling	✅ `ng update` + auto migrations	Codemods	`sv migrate`	Manual / codemods	Manual, mostly smooth
Breaking-change history	AngularJS → Angular rewrite; stable since	Stable	Svelte 4 → 5 big change (runes)	Vue 2 → 3 was painful	Hosting models changed in .NET 8; otherwise stable
Consistency across a team	✅ **Enforced**	❌ Every project differs	Medium	Medium	✅ High
Security (XSS)	✅ Built-in sanitization	✅ Escapes by default	✅ Escapes	✅ Escapes	✅ Escapes; Server mode keeps code off the client
Auth integration	Libraries (MSAL, OIDC)	Libraries	Libraries	Libraries	✅ **ASP.NET Identity, Entra ID, Windows Auth**
i18n	✅ Built in (`@angular/localize`)	react-i18next, react-intl	Paraglide, svelte-i18n	vue-i18n	✅ `IStringLocalizer` / `.resx`
RTL (Persian / Arabic)	✅ CDK `Directionality`, Material RTL	Library-dependent (MUI good)	Manual / CSS	Vuetify, Quasar good	MudBlazor `RightToLeft`; vendors support RTL
Accessibility	✅ CDK a11y tools	Library-dependent	✅ Compiler a11y warnings	Library-dependent	Library-dependent
Offline / PWA	✅ `@angular/pwa`	Workbox / Vite PWA	Vite PWA	Vite PWA	✅ WASM PWA template; Server mode ❌

8. Overall scorecard (1–5)

Criterion	Angular	React	Svelte	Vue	Blazor
Performance	4	4	**5**	**5**	3
Bundle size / first load	3	3	**5**	4	2 (WASM) / 4 (SSR)
Learning curve	2	3	**5**	4	**5 for C# / 2 for JS**
Ecosystem	4	**5**	2	4	3
Enterprise structure	**5**	3	3	3	**5**
Long-term stability / LTS	**5**	4	3	3	**5**
Hiring pool (global)	4	**5**	2	3	2
Full-stack code sharing	1	2 (Node)	2 (Node)	2 (Node)	**5**
Fit: 5 C# devs, internal enterprise apps	3	2	2	3	**5**

Summary

- **Angular:** the best JS choice for large, structured enterprise apps. Everything is built in and LTS is strong, but it's heavy to learn.
- **React:** the biggest ecosystem and hiring pool, and the most freedom. You assemble your own stack.
- **Svelte:** the fastest, smallest and most pleasant to write. The ecosystem is smaller.
- **Vue:** a good balance: approachable and fast, with good official libraries (Router, Pinia).
- **Blazor:** the strongest fit for a team of C# developers. They stay in C#, share models and validation with the backend, and get .NET LTS plus first-class Windows / Entra authentication. Weak spots: the large WebAssembly download, the per-user connection cost in Server mode, and a smaller open-source UI ecosystem (DevExpress, Telerik and MudBlazor cover most of it). For internal apps, Interactive Server or Auto mode avoids most of the download issue.